

SELF-REFINE GUARDRAIL: A TWO-GATE POLICY, CONFIDENTIALITY, AND ERROR VERIFICATION LAYER FOR GENERATIVE AI CUSTOMER SUPPORT CHATBOTS

VIDHURSH KUMAR V, MEENAKSHI SS, JASHAREEN J, and E. RAJALAKSHMI

Department of Artificial Intelligence & Data Science, Rajalakshmi Engineering College, Thandalam, Sriperumbudur, Tamil Nadu, India

Abstract—Large Language Models (LLMs) are increasingly used in customer-support systems, where generated responses can introduce policy violations, disclosure of confidential information, factual errors, and unhelpful or unprofessional communication. This paper presents Self-Refine Guardrail (SRG), a response-level verification layer that separates policy/tone/helpfulness assessment from confidentiality and factual verification. A Redactor produces a sanitized view of the candidate response for Review Agent V1, which evaluates policy compliance, professional tone, and completeness/helpfulness without evaluating confidentiality. Error Checker V2 receives the original response and independently evaluates factual accuracy and confidentiality. The final decision is deterministic: a response is approved only when both gates pass. Responses that fail are sent to a Response Refiner and re-evaluated, with a maximum of two refinement attempts before escalation. The implementation integrates a service-oriented backend with retrieval support and an LLM abstraction layer. Preliminary testing on 20 synthetic query-response cases produced 4 true positives, 16 true negatives, zero false positives, and zero false negatives, corresponding to 100% precision, recall, and F1 score. All 16 initially denied cases were successfully refined within the permitted attempts, with no escalation. These results are preliminary because the evaluation set is small and synthetic.

Index Terms—Large Language Models, AI Guardrails, Policy Compliance, Confidentiality, Error Verification, Retrieval-Augmented Generation, Self-Refinement, Customer Support

I. INTRODUCTION

The deployment of Large Language Models (LLMs) in customer-support applications has made natural-language automation practical at scale. A support model can interpret a customer query, retrieve or infer relevant information, and produce a response in conversational form. The same generative capability, however, creates a verification problem: a response can appear fluent while still violating organizational policy, disclosing confidential information, stating an incorrect fact, or failing to provide useful next steps. Hallucination and reliability surveys identify factual inconsistency as a persistent challenge for generated text [1], while guardrail research emphasizes the need for systematic controls around model inputs and outputs [2], [3].

A response-level guardrail is therefore useful as an intermediate control between response generation and user delivery. A purely rule-based filter can detect explicit patterns such as phone numbers or account identifiers, but it has limited ability to judge contextual policy compliance, tone, or whether a response actually addresses the user's request. Conversely, an LLM used as the sole evaluator can inherit inconsistency, prompt sensitivity, or blind spots associated with model-based judgment [5]–[7]. The design problem is consequently not only detection, but controlled separation of verification responsibilities.

SRG addresses this problem through two deliberately isolated verification gates. The first gate, Review Agent V1, receives a redacted representation of the candidate response and evaluates policy compliance, professional tone, and completeness/helpfulness. Confidentiality is explicitly excluded from V1. The second gate, Error Checker V2, receives the original response and evaluates factual accuracy and confidentiality. This information separation reduces the opportunity for a confidentiality-sensitive evaluator to rely

on a response that has already been transformed for another purpose.

The two gate outputs are combined by a deterministic Boolean rule. If both V1 and V2 pass, the candidate response is approved. If either gate fails, the Response Refiner is invoked to produce a corrected response, after which both gates are rerun. Refinement is bounded to two attempts; persistent failure results in escalation rather than uncontrolled repeated generation. This check–rewrite–recheck design is related to iterative self-refinement [8], but is constrained by explicit safety gates and a fixed retry budget.

The principal contributions are: 1) a two-gate response verification architecture that separates policy/tone/helpfulness evaluation from factual and confidentiality checking; 2) deterministic approval logic that prevents one passing gate from overriding another failing gate; 3) a bounded self-refinement mechanism that attempts correction before escalation; and 4) a preliminary experimental evaluation on a 20-case synthetic test set, including gate-level confusion-matrix metrics and refinement outcomes.

II. RELATED WORK

A) LLM Guardrails and Safeguarding

Dong et al. describe guardrails as mechanisms for constraining undesirable model behavior and argue that their design should be systematic rather than dependent on a single filtering technique [2]. A broader survey of LLM safeguarding covers privacy, attacks, hallucination, and defensive mechanisms [3]. These studies motivate SRG's layered response-verification approach, but SRG narrows the operational focus to customer-support responses and

separates verification responsibilities across independent gates.

B) Retrieval-Augmented Policy Grounding

Retrieval-Augmented Generation (RAG) combines a parametric language model with an external non-parametric knowledge source [4]. In SRG, retrieval is used to ground policy evaluation in organizational material rather than requiring the evaluator to rely only on model parameters. Policy text can be chunked, embedded, indexed, and retrieved as contextual evidence for the policy-compliance stage. The retrieval component supplies evidence; it does not itself constitute the final approval decision.

C) Error Detection and LLM-Based Judgment

Kamoi et al. show that automated detection of errors in generated responses remains challenging and that model-based evaluators can be sensitive to prompting and task characteristics [7]. LLM-as-a-Judge studies similarly identify issues of reliability, consistency, and bias in model-based evaluation [5], [6]. SRG responds by combining semantic evaluation with deterministic confidentiality checking and, more importantly, by making final approval a Boolean combination of two specialized gates rather than an unconstrained evaluator judgment.

D) Iterative Self-Refinement

Self-Refine demonstrates that a language model can improve an initial output by using feedback and iterative rewriting without additional supervised training [8]. SRG adopts the underlying check–feedback–rewrite principle but limits its application to responses that fail verification. The refinement loop is therefore not open-ended: it is invoked only after a failed gate and is terminated after two attempts if the response remains unacceptable.

E) CareGuardAI and Multi-Agent Guardrails

CareGuardAI presents a multi-agent guardrail framework for patient-facing clinical LLMs, combining context-aware screening, safety-constrained generation, dual clinical safety and hallucination risk assessment, and bounded iterative refinement [10]. Its risk-based architecture demonstrates the value of independent safety evaluations before response release. SRG adopts the broader principle of independent verification but targets customer-support response safety rather than clinical risk. In particular, SRG separates the information available to its two evaluators: V1 receives a redacted response for policy, tone, and helpfulness assessment, whereas V2 receives the original response for factual and confidentiality verification. SRG also replaces numerical release thresholds with a deterministic Boolean AND rule and uses a fixed two-attempt refinement budget followed by escalation.

F) Research Gap

The reviewed directions establish strong foundations for guardrails, retrieval, error detection, model-based evaluation, and iterative correction. However, the practical combination of these mechanisms into a response-level customer-support pipeline with explicit information isolation and deterministic multi-gate approval remains an engineering and evaluation challenge. SRG addresses this gap by assigning distinct

evidence views and verification responsibilities to V1 and V2, then applying a fixed decision rule and bounded refinement process.

TABLE I. RELATION OF PRIOR WORK TO SRG

Work / Direction	Primary focus	Relation to SRG
LLM hallucination surveys [1]	Reliability and factual errors	Motivates independent error verification
Guardrail design [2], [3]	Input/output safety controls	Motivates layered response guardrails
RAG [4]	External knowledge grounding	Supports policy-context retrieval
LLM-as-a-Judge [5], [6]	Semantic response evaluation	Supports specialized semantic checks
Error detection [7]	Generated-response errors	Motivates explicit factual verification
Self-Refine [8]	Iterative correction	Provides basis for bounded refinement
SelfCheckGPT [9]	Black-box hallucination checking	Provides related response-level reliability perspective
CareGuardAI [10]	Clinical safety and hallucination guardrails	Related multi-agent safety framework; SRG differs through response-level customer-support verification and information isolation

III. PROBLEM FORMULATION AND DESIGN OBJECTIVES

Let q denote a user query and r denote the candidate response generated by a support LLM. Let $R(r)$ denote the redacted representation supplied to Review Agent V1, and let P denote relevant policy context retrieved for evaluation. V1 produces a binary result $v1$ and V2 produces a binary result $v2$. V1 evaluates policy compliance, professional tone, and completeness/helpfulness using the redacted response and policy context. V2 evaluates factual accuracy and confidentiality using the original response.

$$\text{APPROVED}(r) = V1(R(r), P) \wedge V2(r)$$

The conjunction is intentional. A response that is policy-compliant but confidentially unsafe must not be approved; similarly, a confidentially safe response that violates policy or fails the required quality criteria must not be approved. The decision layer is therefore deterministic even though the individual gate judgments may involve language-model reasoning.

When $\text{APPROVED}(r)$ is false, the Response Refiner receives the candidate response and the available failure feedback. It generates r' and the two gates are evaluated again. If the refined response passes both gates, it becomes the final response. If it continues to fail after two refinement attempts, the case is escalated. This produces a bounded state transition rather than an unbounded generation loop.

The design objectives are consequently: (i) detect confidentiality exposure independently of the policy/tone evaluator; (ii) verify factual correctness on the original response; (iii) ground policy assessment in retrieved organizational context; (iv) require both verification gates to pass; and (v) provide a bounded correction path before escalation.

TABLE II. TWO-GATE DECISION LOGIC

Condition	V1	V2	Final action
Policy/tone/helpfulness acceptable; facts/confidentiality acceptable	Pass	Pass	Approve
V1 failure only	Fail	Pass	Refine / recheck
V2 failure only	Pass	Fail	Refine / recheck
Both gates fail	Fail	Fail	Refine / recheck
Still failing after two attempts	Fail	Fail or unresolved	Escalate

IV. PROPOSED SYSTEM ARCHITECTURE

SRG is organized as a response-verification pipeline rather than as a replacement for the underlying support LLM. The candidate response first enters a Redactor, which creates a sanitized view for V1. The original response remains available to V2 for confidentiality and factual verification. The two evaluations are then combined deterministically. A failed response can enter the Response Refiner and return to the same verification boundary.

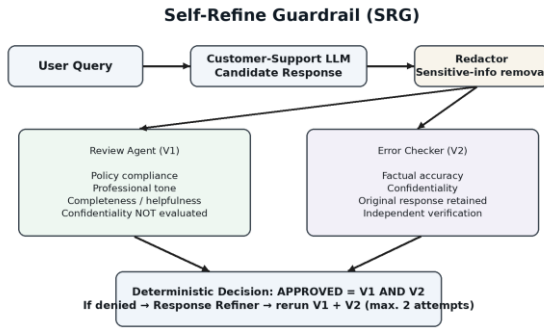


Fig. 1. Final two-gate SRG response-verification architecture.

Fig. 1. Final two-gate SRG response-verification architecture.

A) Redactor

The Redactor transforms the candidate response into the representation consumed by V1. Its purpose is to remove or mask sensitive values before policy, tone, and helpfulness evaluation. The Redactor is not the final confidentiality authority: V2 evaluates the original response independently. This distinction preserves an explicit boundary between a sanitized evaluation view and the source response that must be checked for actual disclosure.

B) Review Agent V1

V1 evaluates three criteria: policy compliance, professional tone, and completeness/helpfulness. Policy compliance is assessed against the relevant policy context, while tone and completeness determine whether the response is professionally framed and provides sufficient guidance to the customer. V1 is explicitly instructed not to evaluate confidentiality. This constraint is central to the architecture because confidentiality is reserved for V2.

C) Error Checker V2

V2 receives the original candidate response rather than the redacted representation. It checks factual accuracy and confidentiality. Confidentiality checking can use deterministic patterns for structured sensitive values and semantic reasoning where contextual interpretation is necessary. By retaining the original response at this gate, the checker can identify information that a redaction stage intentionally removed from the V1 view.

D) Deterministic Decision Layer

The decision layer does not reinterpret the gate outputs. It applies $\text{APPROVED} = V1 \text{ AND } V2$. This removes ambiguity from the final routing condition: one failed gate is sufficient to prevent approval. The decision layer also records which gate failed so that the Response Refiner can target the corrective rewrite.

E) Response Refiner and Escalation

The Response Refiner generates a revised answer using the user query, the candidate response, and the failure feedback. The refined response is not trusted merely because it was rewritten; it must pass both gates again. The retry budget is fixed at two attempts. Persistent failure after the budget is exhausted results in escalation, preventing repeated model generation from bypassing the verification boundary.

V. VERIFICATION METHODOLOGY

A) Policy Retrieval

The policy-grounding component uses a retrieval pipeline in which policy material is divided into text chunks, transformed into sentence embeddings using the all-MiniLM-L6-v2 sentence-transformer model, and indexed with FAISS for similarity search. For a query, the relevant policy chunks are retrieved and supplied as contextual evidence to V1. The retrieval layer supplies evidence; it does not itself constitute the final approval decision. The current implementation uses a top-three retrieval configuration. Retrieval quality is treated separately from the preliminary 20-case gate evaluation and is not claimed as an independently benchmarked result.

B) Information Isolation

The architecture creates two distinct information paths. V1 receives the redacted response plus policy context, while V2 receives the original response. This is not merely an implementation detail: it is a control objective. The policy/tone evaluator is prevented from becoming the confidentiality evaluator, while the error checker retains access to the unmodified candidate needed for disclosure detection.

C) Gate Evaluation Contract

Each gate returns a structured pass/fail result together with a reason suitable for downstream refinement. V1 reasons identify policy, tone, or completeness problems. V2 reasons identify factual or confidentiality problems. The deterministic decision layer combines only the Boolean outcomes. This separation keeps the final approval rule auditable even when the underlying semantic checks are generated by an LLM.

D) Self-Refinement Procedure

The refinement procedure follows four states: initial verification, rewrite, re-verification, and final routing. A response that passes both gates is delivered immediately. A denied response is rewritten and evaluated again. The process stops as soon as both gates pass or when the two-attempt limit is reached.

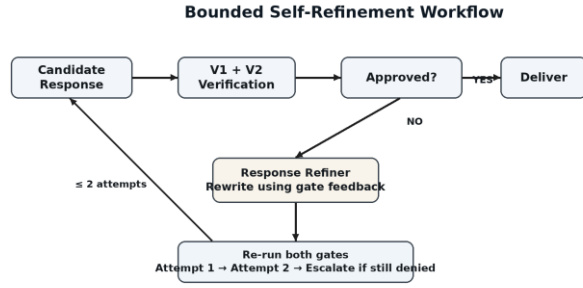


Fig. 2. Bounded check-rewrite-recheck loop with escalation after two unsuccessful

Fig. 2. Bounded self-refinement and escalation workflow.

E) Algorithmic Procedure

The operational procedure can be expressed as follows. Given q and r , generate redacted response $R(r)$; evaluate $V1$ on $R(r)$ and policy context P ; evaluate $V2$ on original r ; compute $A = v1 \wedge v2$; if A is true, return r ; otherwise refine r using failure feedback, increment the attempt counter, and repeat verification while attempts ≤ 2 . If no refined response passes both gates within the budget, route the case to escalation.

$$A_k = V1(R(r_k), P) \wedge V2(r_k), \quad k \in \{0, 1, 2\}$$

The formulation emphasizes that every refined candidate is subject to the same acceptance condition. Refinement therefore changes the response content, not the safety contract.

VI. IMPLEMENTATION AND SYSTEM INTEGRATION

SRG is implemented as a modular backend whose guardrail logic is separated from the model interface. The current application stack uses a FastAPI service, n8n orchestration, sentence-transformer embeddings with FAISS for policy retrieval, and a Groq LLM client using the openai/gpt-oss-20b model. These components provide the execution surface around the core verification logic while keeping the two-gate safety contract independent of the chatbot interface.

A) LLM Abstraction

An LLM-client abstraction separates model calls from the guardrail components. The current implementation uses a Groq LLM client with the openai/gpt-oss-20b model, allowing Review Agent V1, Error Checker V2, and the Response Refiner to invoke the model through a common interface. This separation keeps model-specific integration outside the deterministic verification and routing logic.

B) Service and Orchestration Layer

FastAPI provides the backend service boundary, while n8n is used to orchestrate the request flow between the chatbot and the SRG backend. The current workflow uses a webhook to forward the user input to the FastAPI /chat endpoint and return the resulting SRG decision and final response. This separation keeps the guardrail logic inside the backend while allowing the workflow to connect with the customer-support front end.

C) Module Responsibilities

TABLE III. SRG MODULE RESPONSIBILITIES

Module	Input	Primary responsibility	Output
Redactor	Candidate response	Create sanitized evaluation view	Redacted response

Review Agent V1	Redacted response + policy	Policy, tone, completeness/helpfulness	V1 pass/fail + reason
Error Checker V2	Original response	Factual accuracy + confidentiality	V2 pass/fail + reason
Decision layer	V1, V2	Deterministic AND rule	Approve / deny
Response Refiner	Query + response + feedback	Correct failed response	Refined response
Policy Retriever	Query/policy store	Retrieve relevant policy context	Policy chunks

D) Operational Traceability

A practical execution record should retain the candidate response identifier, gate outcomes, failure reasons, refinement attempt count, final approval state, and escalation state. Such a record supports debugging and experimental analysis without changing the deterministic decision rule. The experimental results reported in the next section are derived from the recorded 20-case test output rather than from design-time expectations.

The deterministic portions of SRG introduce bounded processing relative to response length. Redaction and structured confidentiality detection operate over the response text, while semantic gate evaluation and refinement are dominated by LLM inference latency. Policy retrieval adds embedding and similarity-search overhead. Because the refinement budget is fixed at two attempts, the number of additional generation-and-verification cycles is also bounded. These are design characteristics rather than measured latency results; no end-to-end latency benchmark is claimed in the present evaluation.

E) Complexity and Operational Considerations

VII. OPERATIONAL THREAT MODEL AND WORKED EXAMPLE

A) Threat Model

SRG considers several classes of unsafe interaction relevant to customer-support systems. A direct prompt-injection attempt may ask the support model to ignore prior instructions or policy constraints. A policy-circumvention request may seek an unauthorized exception or commitment. A confidentiality attack may attempt to induce the model to repeat a customer's account identifier, phone number, email address, or other sensitive value. A combined attack may contain both a policy violation and a disclosure request. Finally, repeated probing may attempt to exploit the refinement loop by causing repeated rewrites until an unsafe response is produced.

TABLE VII. THREAT CATEGORIES AND SRG CONTROLS

Threat category	Representative behavior	SRG control
Prompt injection	Ignore previous policy or instructions	Policy evaluation + deterministic approval
Policy circumvention	Request unauthorized commitment	V1 policy-compliance check
Confidentiality exposure	Repeat account/PII data	V2 confidentiality check on original
Factual error	State an incorrect support fact	V2 factual-accuracy check
Refinement abuse	Repeatedly probe rewrite loop	Maximum two attempts + escalation

The two-gate architecture addresses these threats through distinct control points. Policy and response-quality concerns are evaluated by V1 using the redacted view and policy context, while confidentiality and factual correctness are evaluated by V2 on the original response. The deterministic AND rule prevents a passing V1 result from overriding a failing V2 result. The two-attempt refinement limit constrains repeated generation.

B) Worked End-to-End Example

Because V2 fails, the deterministic approval condition is false even if V1 passes. The response is therefore sent to the Response Refiner with the failure reason. The refiner can preserve the useful refund information while removing the sensitive value or correcting any identified policy issue. The revised response is then evaluated again by both gates. If both pass, the revised response is approved; otherwise, the process continues only within the two-attempt limit.

Consider a customer query requesting confirmation of a refund together with a registered phone number. The support LLM produces a candidate response containing the requested phone number and a refund statement. The Redactor removes the sensitive phone value from the V1 representation. V1 then checks the remaining response for policy compliance, professional tone, and completeness. Independently, V2 receives the original response and detects the confidential phone value while checking the factual claim.

VIII. EXPERIMENTAL EVALUATION

A) Dataset and Test Setup

The preliminary evaluation uses a synthetic dataset of 20 query-response test cases designed to exercise clean responses, policy violations, confidentiality violations, tone or completeness failures, combined failures, refinement cases, and a case intended to remain unsafe or require escalation. Each case records the user input, draft response, policy context, expected approval state, and expected failure gate. The evaluation focuses on the final gate decision and the behavior of the bounded refinement loop.

The 20 cases are distributed as follows: four clean/compliant cases; four policy-violation cases; four confidentiality-violation cases; three tone/completeness failures; two cases combining policy and confidentiality problems; two cases expected to become safe after refinement; and one case intended to remain unsafe or require escalation. This distribution is used only to characterize the tested scenarios; it is not intended to represent a production traffic distribution.

TABLE IV. PRELIMINARY TEST-CASE DISTRIBUTION

Category	Cases	Purpose
Clean / compliant	4	Verify approval of acceptable responses
Policy violation	4	Exercise policy gate failures
Confidentiality violation	4	Exercise confidentiality checking
Tone / completeness	3	Exercise response-quality failures
Policy + confidentiality	2	Exercise simultaneous failures
Safe after refinement	2	Verify correction and recheck
Remain unsafe / escalate	1	Exercise persistent-risk path

B) Gate Performance

Across the 20 test cases, the recorded confusion-matrix counts are $TP = 4$, $TN = 16$, $FP = 0$, and $FN = 0$. The resulting precision, recall, and F1 score are each 100.00%, and the false-negative rate is 0.00%. These values describe the observed test run on the supplied synthetic cases; they should not be interpreted as estimates of performance on a broader population.

TABLE V. GATE PERFORMANCE RESULTS

Metric	Result
Test cases (N)	20
True positives (TP)	4
True negatives (TN)	16
False positives (FP)	0
False negatives (FN)	0
Precision	100.00%
Recall	100.00%
F1 score	100.00%
False-negative rate	0.00%

For unsafe-response identification, precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and the false-negative rate is $FN/(FN+TP)$. With four positive cases and no false positives or false negatives in this test run, all three primary classification measures attain 100%. The result is internally consistent with the supplied test output.

C) Self-Refinement Performance

Sixteen of the 20 cases were initially denied by the verification process. All 16 subsequently produced an approved response through the bounded refinement loop, giving a refinement success rate of 100.00%. No case reached escalation in the recorded run, corresponding to an escalation rate of 0.00%. The successful cases include responses initially denied for tone or completeness, policy and confidentiality issues, and other gate failures.

TABLE VI. SELF-REFINEMENT RESULTS

Self-refinement measure	Result
Initially denied	16
Successful refinements	16
Refinement success rate	100.00%
Escalations	0
Escalation rate	0.00%

D) Case-Level Observations

The detailed test output illustrates several representative behaviors. TC13 was initially rejected by V1 for unprofessional communication and lack of useful next steps while V2 passed; one refinement produced an approved response. TC14 and TC15 similarly failed V1 because the responses did not provide sufficient actionable guidance or clear return/eligibility information, and both were approved after one refinement.

TC16 and TC17 demonstrate combined gate failures. TC16 was rejected for a policy violation at V1 and exposure of a confidential account number at V2; TC17 failed V1 for an out-of-policy refund condition and V2 for exposure of a confidential registered email. Both were corrected in one refinement attempt. TC18–TC20 similarly show initially denied responses involving policy, confidentiality, or unsafe disclosure conditions, followed by successful refinement in the recorded test output.

E) Results Summary

The experimental run therefore shows two distinct properties. First, the deterministic two-gate decision produced no observed classification errors on the 20 supplied cases. Second, the refinement loop converted every initially denied case into an approved result within the permitted retry budget. Because the cases are synthetic and limited in number, the results demonstrate successful behavior on the constructed test set rather than statistical evidence of generalization.

IX. DISCUSSION

The most important design observation is the separation of responsibilities. V1 is intentionally prevented from evaluating confidentiality, while V2 receives the original response and evaluates confidentiality and factual correctness. This means the final approval decision does not depend on a single evaluator having simultaneous responsibility for every safety property. The AND rule also makes the final control condition simple to inspect and reproduce.

The preliminary results are consistent with the intended behavior of the refinement loop. Cases rejected because of response quality or safety issues were not automatically discarded; instead, failure feedback was used to generate a revised candidate and the same two gates were reapplied. The fact that all 16 initially denied cases were approved after refinement in this test run indicates that the constructed scenarios were generally correctable within one or two attempts.

The role of retrieval should be interpreted separately from the observed classification metrics. The source development document identifies sentence-transformer and FAISS policy retrieval as part of the intended implementation, but its retrieval functionality was not independently evaluated at the earlier development stage. Consequently, the reported 20-case metrics should not be interpreted as a benchmark of retrieval quality.

Likewise, the evaluation does not establish that the underlying semantic evaluators are universally reliable. The literature identifies known limitations in LLM-based judgment and automated error detection [5]–[7]. SRG’s architectural response is to constrain those judgments with explicit responsibilities and deterministic aggregation, but the effectiveness of that strategy requires broader testing with real model behavior, varied prompts, adversarial cases, and larger datasets.

X. LIMITATIONS AND EVALUATION BOUNDARY

The present evaluation has several limitations. First, the dataset contains only 20 synthetic cases. Such a small constructed set is useful for functional verification but cannot support statistically conclusive claims about production performance, robustness, or generalization. Second, the reported run does not constitute a broad latency benchmark. Third, the effectiveness of policy retrieval is not separately quantified. Fourth, semantic evaluation behavior can vary with the underlying LLM and prompt configuration.

The confidentiality detector is also inherently dependent on the patterns and semantic mechanisms used to identify

sensitive information. Structured identifiers are amenable to deterministic detection, but novel formats, contextual disclosures, and obfuscated values can be more difficult. This motivates retaining an independent V2 check rather than treating redaction itself as proof that a response is safe.

Finally, successful refinement in the present test set should not be interpreted as evidence that every unsafe response can be repaired automatically. A robust deployment must preserve escalation for persistent or ambiguous failures. The two-attempt limit is therefore a safety boundary as well as a performance constraint.

XI. CONCLUSION

This paper presented Self-Refine Guardrail (SRG), a response-level safety layer for generative AI customer-support systems. The final architecture separates policy, tone, and helpfulness evaluation from factual and confidentiality verification through two gates: Review Agent V1 operates on a redacted response, while Error Checker V2 evaluates the original response. A deterministic AND rule requires both gates to pass before delivery, and a bounded Response Refiner provides up to two correction attempts before escalation.

On the supplied 20-case synthetic evaluation, SRG recorded 4 true positives, 16 true negatives, zero false positives, and zero false negatives, yielding 100.00% precision, recall, and F1 score with a 0.00% false-negative rate. Sixteen cases were initially denied, all 16 were successfully refined, and no escalation occurred in the recorded run. These findings are preliminary and limited to the constructed test set.

The principal next validation step is broader experimentation with real LLM behavior, larger and more diverse datasets, adversarial prompts, independently measured retrieval quality, and latency benchmarking. The architectural emphasis remains unchanged: isolate verification responsibilities, require deterministic agreement for approval, and preserve a bounded path to human escalation when automatic correction is insufficient.

REFERENCES

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, Art. no. 42, pp. 1–55, 2025.
- [2] Y. Dong, R. Mu, G. Jin, Y. Qi, J. Hu, X. Zhao, J. Meng, W. Ruan, and X. Huang, “Position: Building Guardrails for Large Language Models Requires Systematic Design,” in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235, pp. 11375–11394, 2024.
- [3] Y. Dong, R. Mu, Y. Zhang, S. Sun, T. Zhang, C. Wu, G. Jin, Y. Qi, J. Hu, J. Meng, S. Bensalem, and X. Huang, “Safeguarding large language models: A survey,” *Artif. Intell. Rev.*, vol. 58, Art. no. 382, 2025, doi: 10.1007/s10462-025-11389-2.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023.

